# AquaFlow - A1 Grade Checklist

> Derived directly from the professor's marking criteria lecture. Each item maps to a specific graded criterion. **All boxes must be checked for A1.**

---

## 1. Code Structure - SOLID Principles

> Professor: *"A is really that you have thought through this, understood the right choices, and applied these professionally."* Global variables, no SOLID evidence → D/E. No evidence at all → F.

### S - Single Responsibility Principle

- [ ] **Every class has exactly ONE clearly-defined responsibility.** No "god classes" doing multiple jobs (e.g., a class that both reads the sensor AND controls the pump AND logs simultaneously).
- [ ] **Confirm current classes are properly separated:** `GestureSensor`, `PumpController`, `FlowMeter`, `LcdDisplay`, `FillingController`, `Monitor`, `Timer`, `Logger` — each should own one concern only.
- [ ] **Document in README/docs** which responsibility each class owns (justifies SRP compliance to the marker).

### O - Open/Closed Principle

- [ ] **Base classes exist that can be extended without modification.** e.g., `IHardwareDevice` as the base - new sensors should be addable by inheritance, not by editing existing classes.
- [ ] **Check** `IHardwareDevice.h` - does it provide a solid enough base that `FlowMeter`, `PumpController` etc. inherit from it without modifying the base?
- [ ] **Use inheritance + virtual functions** to add functionality (e.g., a future `4-channel ADC` class extends a base `ADC` class).
- [ ] **Document your inheritance hierarchy** - show it in a UML diagram.

### L - Liskov Substitution Principle

- [ ] **Base class interfaces are future-proof.** If you swap a derived class for the base class, nothing breaks.
- [ ] **Specifically: callbacks/interfaces in base classes must be flexible enough** to accommodate future derived classes (e.g., use `std::vector<float>` instead of a single `float` in callbacks so multi-channel readings don't break the interface).
- [ ] **Check** `IHardwareDevice.h` - if it declares a `getData()` callback with a fixed return type, verify the derived classes don't break substitutability.

- ☐ **Document in code comments/docs** your reasoning on Liskov compliance or why you consciously relaxed it.

### I - Interface Segregation Principle

- ☐ **No bloated interfaces.** Each class/callback interface exposes ONLY what the consumer needs - not a massive omnibus interface.
- ☐ **Sensor-specific callbacks are separate** - e.g., a flow reading callback is distinct from a gesture event callback; they're not crammed into one generic handler.
- ☐ **Verify** that no single class is being used as a catch-all for unrelated data.

### D - Dependency Inversion Principle

- ☐ **Decision documented:** Professor explicitly said DIP is hard in C++ and is optional - but your **choice must be documented** (in README/docs) explaining why you did or didn't apply it.
- ☐ **(Optional, high marks):** If attempting DIP, use C++ templates to decouple high-level logic from low-level hardware drivers so either can be swapped independently.
- ☐ **At minimum:** ensure high-level modules ( `FillingController` ) depend on **abstractions** ( `IHardwareDevice` ), not concrete classes directly.

---

## 2. Encapsulation

> Professor: *"A = clearly defined public interface, all variables in private, access only via getters/setters/callbacks, efficient internal data structures."* Global variables → D/E instantly. Everything dumped in `main` → F.

- ☐ **Zero global variables.** Do a full search across all `.cpp` and `.h` files. Any global mutable state is an immediate D/E.
- ☐ **All member variables are** `private` (or `protected` where justified). No `public` data members.
- ☐ **All data access is through getters, setters, or callbacks only.** No direct member access from outside the class.
- ☐ **Internal data structures are efficient** - e.g., consider using `std::atomic` for shared sensor readings, ring buffers or double-buffering for high-frequency data.
- ☐ `main.cpp` **contains only initialisation code** - no real-time logic, no loops, no processing. After init, it simply blocks on `sigwait()`.
  - **Current state:** `main.cpp` already does this correctly with `sigwait()` - confirm it stays this way.
- ☐ **All callbacks are transmitted through** `std::function` **or abstract interface** - not via raw global function pointers.

- [ ] **Safe data receiving and releasing** - verify fault-checking mechanisms are in place when sensors disconnect or provide erroneous data.

---

## 3. Memory Management

> Professor: *"Use STL. Use smart pointers. Never void pointers. Never malloc. Prefer copy constructors."* `void*` or `malloc` = "deal-breaker" for this criterion.

- [ ] **No raw `new` / `delete` unless absolutely required** (e.g., Qt forces it). Use RAII.
- [ ] **No `malloc` / `free` anywhere in the codebase.** Run a search.
- [ ] **No `void*` pointers anywhere.** Run a search.
- [ ] **Use `std::shared_ptr` / `std::unique_ptr`** for any dynamically allocated objects.
- [ ] **Prefer copy constructors / value semantics** - assign objects directly (e.g., `std::thread thr = std::thread(...)`) instead of heap allocation.
- [ ] **Use STL containers** (`std::vector`, `std::queue`, `std::deque`) instead of C-style arrays where possible.
- [ ] **Review Rule of 3 / Rule of 5** - if any class manages a resource (thread, file handle, socket), ensure copy constructor, copy assignment, and destructor are properly defined (or explicitly deleted).
- [ ] **No memory leaks** - verify all threads are joined, all connections closed, all resources released in `shutdown()` methods.

---

## 4. Real-Time Coding

> Professor: *"This is the most important thing - and the easiest way to get marks."* Arduino-style polling loops, `sleep()`-based timing, or a frozen UI = automatic fail for this criterion.

### 4a. Timing Requirements - DOCUMENT THIS (easy marks, often missed!)

- [ ] **State your system's timing requirements explicitly in the README or docs.** e.g.:
  - *"The pump must respond to a cup-detected event within 50 ms of a proximity event."*
  - *"The flow meter samples at interrupt-driven rate; callback must complete in < 10 ms."*
  - *"The GestureSensor polls the APDS-9960 FIFO every 50 ms via timerfd."*
  - *"The state-machine tick fires every 100 ms via timerfd."*
  - Professor says ~30% of teams miss this and "earn a straight fail for it" — **do not skip**.

### 4b. Event-Driven, Non-Blocking Architecture

- ☐ **No polling loops.** No `while(true) { check_sensor(); sleep(100ms); }` patterns.
- ☐ **All I/O is event-driven:** threads block on a hardware event (GPIO interrupt, timerfd, I2C data-ready pin) and wake up only when data arrives.
- ☐ `Timer` **class uses** `timerfd` (or equivalent blocking mechanism) - not a `sleep()` loop.
  - **Current state:** `main.cpp` uses `loopTimer.registerCallback()` - confirm `Timer` internally uses `timerfd` or `std::condition_variable`, not `sleep()`.
- ☐ `FlowMeter` **uses GPIO interrupt** on the pulse pin — not polling.
- ☑ `GestureSensor` **uses** `timerfd` **blocking read** — replaced `sleep_for(50ms)` polling with a kernel-managed timerfd. Zero CPU between firings. ✅ Done.
- ☐ **No** `sleep()` **or** `usleep()` **used to establish timing.** These are unreliable on Linux (multitasking OS). Search the codebase.

### 4c. Fast & Deterministic Callbacks

- ☐ **Callbacks in sensor drivers are fast and deterministic.** No heavy computation, no I/O, no blocking calls inside a callback.
- ☐ **Slow/unpredictable work (e.g., any processing logic) is offloaded** to a separate `std::thread` - the callback just queues the data and returns.
- ☐ `FillingController::tick()` (called from timer callback) must be deterministic - no unbounded loops or I/O inside it.

### 4d. Thread Architecture

- ☐ **Each blocking I/O operation runs in its own dedicated worker thread:**
  - `Timer` worker thread (timerfd → `FillingController::tick()` callback)
  - `FlowMeter` worker thread (GPIO interrupt → pulse count, `atomic<int>`)
  - `GestureSensor` worker thread (timerfd → I2C FIFO read → proximity callback → `atomic<bool>` in `FillingController`)
- ☐ `main()` **thread only sleeps** (via `sigwait`) - it does not participate in any real-time processing.
- ☐ **Thread-safe data sharing** - use `std::atomic` or `std::mutex` + `std::lock_guard` for any data accessed from multiple threads.
- ☐ **No dead locks** - verify no two mutexes are locked in opposite orders in different threads.

---

## 5. Revision Control & Project Management

> Professor: *"Last-minute single upload = very low marks. Only one person committing = visible, penalised."*

### 5a. Git Commit & Release Quality

- ☐ **GitHub Release created by the deadline** - the professor will download and mark the latest *release*, not just the latest pushed commit. Don't forget to cut a tag/release!
- ☐ **Every commit has a descriptive message** - not "updated file", "fix", or "changes". Each message should explain WHAT was changed and WHY.
- ☐ **Commits reference issues where applicable** - e.g., "Fixed flow meter callback race condition (closes #12)".
- ☐ **Use git from the command line** (not just GUI drag-and-drop) to ensure full, detailed commit messages.
- ☐ **Commits are frequent and incremental** - showing a clear development history, not a single bulk upload at the end.

### 5b. Branching Strategy

- ☐ **Feature branches are used** - each feature/component developed on its own branch.
- ☐ **Branches are merged into** `main` via pull requests (or at least documented merges).
- ☐ **GitHub's commit graph** shows a visible branching and merging history (professor says "I look at this graph").
- ☐ **All team members have committed** - the Git log must show contributions from multiple people.

### 5c. Project Management Evidence

- ☐ **Use GitHub Issues or GitHub Projects** to track work items / tasks.
- ☐ **Issues reference commits** - ("closes #X" in commit messages links the commit to the issue).
- ☐ **Sprint-based or milestone-based planning visible** - show at least weekly progress updates or milestones.
- ☐ **All team members are visible as contributors** in the project management tool.
- ☐ **Clear division of labor marked** - each group member's specific area of responsibility must be clearly documented in the README or final report.

---

## 6. Documentation, Testing & PR

### 6a. Unit Tests (CMake `make test`)

- ☐ **Unit tests exist for each major class** (not just integration tests).
- ☐ `CMakeLists.txt` **has** `enable_testing()` **and** `add_subdirectory(tests)`**.**
- ☐ **Running** `make test` **from the build directory executes all tests automatically.**
- ☐ **Tests are meaningful** - they send known inputs and verify expected outputs (e.g., send fake pulse counts to `FlowMeter`, verify `getVolumeML()` returns the correct value).

- ☐ **(Optional, high marks):** Use **Google Test (** `gtest` **)** framework for structured, industry-standard unit tests.
- ☐ **Test coverage includes:** `FlowMeter`, `PumpController`, `FillingController` state machine transitions, `Timer` callback firing.

### 6b. README / Installation Documentation

- ☐ **Prerequisites listed** - exact `apt-get install` commands needed (e.g., `libgpiodcxx`, `cmake`).
- ☐ **Build instructions** - exact `cmake .. && make` commands.
- ☐ `make test` **instructions** - how to run unit tests.
- ☐ **Bill of Materials (BoM) under £75** - list every hardware component with cost and explicitly verify the total is under the £75 budget limit.
- ☐ **Hardware wiring diagram** - circuit diagram or clear wiring table ( partial - README has wiring tables, add a proper circuit diagram image).
- ☐ **Installation on fresh Raspberry Pi** - step-by-step from a blank Pi to running system.
- ☐ **Timing requirements documented** (see 4a - this overlaps with real-time criterion).

### 6c. Code-Level Documentation

- ☐ **All public class members documented** - every public method, callback, and data member has a comment explaining its purpose, parameters, and return value.
- ☐ **(Recommended):** Use **Doxygen-compatible comment syntax** ( `/** @brief ... @param ... @return ... */` ) so documentation can be auto-generated.
- ☐ **Callback signatures documented** - explain exactly what each callback delivers and when it fires.
- ☐ **Design decisions documented** - the `docs/decisions.md` (already exists ) should explain SOLID choices, thread design decisions, and any trade-offs.

### 6d. UML / Architecture Diagrams

- ☐ **UML class diagram** showing all classes and their relationships (inheritance, composition, dependencies).
- ☐ **Timing/sequence diagram** showing the temporal flow of events (e.g., cup detected → gesture event → state machine → pump on → flow meter pulses → pump off).
- ☐ **System architecture diagram** showing how hardware components connect to software classes.
- ☐ Include these diagrams in the README or a dedicated `docs/architecture.md`.

### 6e. PR / Social Media & Open Source

- ☐ **Demo video** - a short YouTube/video clip showing the system working end-to-end.
- ☐ **Project logo/branding** (professor highlighted the Brew Genie team for this).
- ☐ **(Stretch):** Technical blog post or write-up (professor mentioned RS Design Spark article).

- ☐ Link any social/PR content from the README.
- ☐ **Project Licensing** - ensure your repository has an open-source license file (e.g., MIT, GPL), as it's evaluated under the Promotion criteria.

---

## Deal-Breakers (Automatic Fail / Zero for That Criterion)

> These will kill your grade. Verify **none** of these apply.

- ☐ **Not using C++** - the project MUST be C++. (already C++)
- ☐ **Program becomes unresponsive** under normal operation - e.g., UI freezes during processing.
- ☐ `sleep()` / `wait()` **used for timing** instead of event-driven architecture.
- ☐ **Single massive polling loop** (Arduino `loop()` style).
- ☐ **Global variables** used anywhere.
- ☐ `void*` **pointers or** `malloc()` anywhere in the code.
- ☐ **No revision control history** - only one commit at the very end.
- ☐ **Only one team member committing** - all members must have visible contributions.
- ☐ **Trivial project scope** - minimal code, mostly PR fluff (not a concern for AquaFlow, but worth confirming sufficient code depth).
- ☐ **Not plotting values on screen** - technical requirements explicitly state the system must measure values AND *plot them*.
- ☐ **No mouse interaction** - technical requirements explicitly state the UI must allow *mouse interaction* to change parameters.
- ☐ **Not running as a standalone embedded app** - the system MUST boot up automatically as a standalone application on the Raspberry Pi without manual terminal interference.

---

## Quick-Wins (Highest ROI, Easiest to Miss)

These are the things the professor explicitly says teams consistently skip and "earn a straight fail" for:

1. ** Write the timing requirements down** - literally one paragraph in the README stating response times.
2. ** Improve commit messages** - run through your Git log and ensure no "updated file" messages exist.
3. ** Run `make test` ** - ensure tests execute cleanly from a fresh build.
4. ** Add a UML diagram** - even a simple hand-drawn one photographed and uploaded.

5. ** Record a demo video** - even a 60-second phone recording of the dispenser working.